

Sistema de Processamento Paralelo de Requisições a um Banco de Dados

Avaliação M1 — IPC, Threads e Paralelismo — Sistemas Operacionais

Universidade do Vale do Itajaí (Univali) — Setembro de 2026

1. Identificação

Trabalho: Avaliação M1 — IPC, Threads e Paralelismo, disciplina de Sistemas Operacionais, Universidade do Vale do Itajaí (Univali).

Autores:

Eduardo Mateus Ferreira
[NOME COMPLETO 2] — [RA]
[NOME COMPLETO 3] — [RA]

Repositório público do código: <https://github.com/eCanoppy/sistema-bd-paralelo>

2. Enunciado do projeto

O trabalho consiste em desenvolver um sistema que simula o funcionamento interno de um gerenciador de requisições a um banco de dados, com múltiplos processos e threads, atendendo aos requisitos técnicos obrigatórios definidos pelo professor:

- Cliente e servidor executando como **processos de sistema operacional distintos** (não threads de um único processo simulando dois papéis).

- Comunicação entre eles por **IPC real** — neste trabalho, FIFO (named pipe) POSIX — nunca por variável global, arquivo lido por polling ingênuo, ou chamada de função direta.
- O servidor processando requisições em paralelo através de um **pool de múltiplas threads** (`pthread_create`).
- Acesso à tabela compartilhada entre as threads protegido por **exclusão mútua real** (`pthread_mutex_t`).
- Suporte às operações `INSERT` , `SELECT` , `UPDATE` e `DELETE` sobre um registro simples { `id` , `nome` } .

3. Contexto da aplicação

Um banco de dados real precisa atender múltiplas requisições concorrentes sem corromper seus próprios dados nem serializar tudo numa única thread (o que desperdiçaria os núcleos disponíveis da máquina). Este trabalho simula essa arquitetura em escala reduzida:

- O **processo cliente** (`bin/cliente`) representa uma aplicação (ou um usuário) que dispara uma requisição de banco e espera a resposta. Cada execução do binário é uma requisição isolada — o mesmo padrão de um script/aplicação que abre uma conexão curta, faz uma operação e fecha.
- O **processo servidor** (`bin/servidor`) representa o gerenciador de banco: recebe requisições de qualquer número de clientes através de um canal IPC único e as distribui para um pool fixo de threads trabalhadoras, que competem pelo acesso à tabela compartilhada.

O ponto central do trabalho — e o que a disciplina de Sistemas Operacionais está avaliando — não é a funcionalidade de CRUD em si (trivial), mas a **correção sob concorrência real**: duas threads não podem, ao mesmo tempo, escrever no mesmo slot do vetor e corromper o estado; duas requisições concorrentes de `INSERT` com o mesmo `id` não podem ambas ter sucesso.

3.1 Arquitetura escolhida

- **IPC:** um FIFO fixo (`bd_requisicoes.fifo`) recebe todas as requisições de todos os clientes, no formato texto `pid|seq|OP|id|nome` . Cada cliente cria seu próprio FIFO de resposta (`bd_resposta_<PID>.fifo`), evitando que respostas de clientes diferentes se cruzem.
- **Concorrência no servidor:** uma thread principal lê o FIFO de requisições e enfileira cada uma numa fila limitada (mutex + variáveis de condição); um **pool fixo de N threads trabalhadoras** (`pthread_create` na inicialização, N configurável por argumento de linha de comando) consome dessa fila e executa a operação sobre o banco.
- **Banco simulado:** um vetor `Registro banco[MAX_REGISTROS]` em memória do processo servidor, protegido por um único `pthread_mutex_t` , com persistência em `banco.txt` (reescrito após cada operação de escrita) para sobreviver a reinicializações do servidor.

4. Resultados obtidos com as simulações

Antes das medições finais de desempenho, o fluxo funcional completo foi validado ponta a ponta (`testes/smoke_e2e.sh`): sequência `INSERT` → `SELECT` → `UPDATE` → `SELECT` → `DELETE` → `SELECT` , confirmando que cada operação retorna o status e o payload esperados, e que o `banco.txt` reflete corretamente o estado final (vazio, após o único registro ter sido removido).

A prova de ausência de corrupção sob concorrência real foi feita em dois níveis:

1. **Nível de unidade** (`testes/test_banco.c`): 8 threads inserindo 100 registros cada (800 inserções concorrentes, IDs não sobrepostos) diretamente contra o módulo `banco.c` , sem passar pelo IPC — isola a garantia do mutex da complexidade do FIFO. Resultado: `total_ocupados == 800` , todos os registros amostrados íntegros.
2. **Nível de integração** (`testes/teste_concorrencia.sh`): até 100 processos-cliente reais disparados em paralelo contra o servidor de verdade, via FIFO, com pool de até 8 threads — ver Seção 6.

Durante essa segunda rodada de testes foi encontrado e corrigido um bug real de concorrência na camada de IPC (não na exclusão mútua sobre o banco):

descrito na Seção 6.1.

5. Códigos importantes de implementação

5.1 Exclusão mútua sobre a tabela compartilhada (`src/banco.c`)

Toda operação de leitura/escrita no vetor compartilhado adquire o mesmo mutex antes de tocar os dados e o libera antes de retornar — inclusive em cada caminho de erro (registro não encontrado, banco cheio), evitando qualquer seção crítica destravada por engano:

```
int banco_inserir(Banco *banco, int id, const char *nome) {
    pthread_mutex_lock(&banco->mutex);
    int resultado = -1;
    if (indice_do_id(banco, id) == -1) {
        int slot = primeiro_slot_livre(banco);
        if (slot != -1) {
            banco->registros[slot].id = id;
            strncpy(banco->registros[slot].nome, nome, MAX_NOME - 1);
            banco->registros[slot].nome[MAX_NOME - 1] = '\0';
            banco->registros[slot].ocupado = 1;
            banco->total_ocupados++;
            resultado = 0;
        }
    }
    pthread_mutex_unlock(&banco->mutex);
    return resultado;
}
```

A verificação de duplicidade (`indice_do_id`) e a escrita do novo registro acontecem **dentro** da mesma seção crítica — se essas duas etapas estivessem em travas separadas, duas threads poderiam passar pela checagem de "id livre" antes de qualquer uma escrever, e ambas inserirem o mesmo `id` (condição de corrida clássica de "check-then-act").

5.2 Pool de threads e fila produtor/consumidor (`src/servidor.c`)

A thread que lê o FIFO nunca executa a operação de banco diretamente — ela só enfileira; quem processa são as N threads do pool, cada uma bloqueada em `pthread_cond_wait` até haver trabalho:

```
static void *worker_thread(void *arg) {
    ContextoWorker *ctx = (ContextoWorker *)arg;
    Requisicao req;
    while (fila_desenfileirar(ctx->fila, &req) == 0) {
        Resposta resp;
        processar_requisicao(ctx->banco, &req, &resp);
        if (req.op != OP_SELECT) {
            banco_persistir(ctx->banco, CAMINHO_BANCO);
        }
        enviar_resposta(req.pid_cliente, &resp);
    }
    return NULL;
}
```

5.3 Handshake de IPC entre cliente e servidor

O ponto mais delicado do trabalho não foi o mutex (razoavelmente padrão), mas a sincronização correta de abertura dos FIFOs — coberto em detalhe nas Seções 6.1 e 6.2.

6. Resultados obtidos com a implementação

6.1 Bug de concorrência real encontrado durante os testes

A primeira tentativa de medir desempenho com muitos clientes simultâneos (a partir de ~90 processos-cliente concorrentes) revelou um bug: o servidor descartava partes de mensagens no meio de uma requisição. A causa raiz não estava na exclusão mútua sobre o banco (que nunca corrompeu dado em nenhum teste), e sim na camada de remontagem de mensagens do FIFO: o

buffer usado para acumular bytes lidos do `read()` até encontrar um `\n` tinha 256 bytes — dimensionado para o tamanho de **uma** linha — mas, sob alta concorrência, um único `read()` do servidor podia trazer **várias** requisições de clientes diferentes coladas, e o acúmulo de linhas ainda não processadas excedia esse limite. A checagem de overflow então descartava o buffer inteiro, inclusive o início de uma mensagem que continuaria no próximo `read()`, produzindo uma linha corrompida como o exemplo `1|INSERT|96|Cliente96` (faltando o campo de PID do cliente no início).

Correção: o buffer de reconstrução de linha foi ampliado para 8 KB, dimensionado pelo volume esperado de mensagens em trânsito sob rajada, não pelo tamanho de uma mensagem individual. Após a correção, a mesma carga (100 clientes concorrentes) rodou sem nenhuma mensagem descartada — ver tabela na Seção 6.3.

Esse achado é, por si, um resultado relevante do trabalho: mostra que testar unicamente com poucos clientes (como o fluxo `smoke_e2e.sh` da Seção 4) não é suficiente para expor bugs de concorrência — eles só aparecem sob volume real, reforçando por que a avaliação exige prova de funcionamento sob carga, não apenas um exemplo feliz.

6.2 Corrida na abertura do FIFO de resposta

Um segundo problema, encontrado antes deste, foi uma corrida clássica de FIFO: o cliente abria seu FIFO de resposta em modo leitura e então chamava `read()` bloqueante, esperando (incorretamente) que essa chamada ficasse bloqueada até o servidor abrir o outro lado para escrita. Pela semântica POSIX de pipes/FIFOs, `read()` bloqueante retorna EOF (`0`) **imediatamente** se, no instante exato da chamada, não houver nenhum processo com o FIFO aberto para escrita — ele não espera um escritor futuro aparecer. Como o servidor só abre o FIFO de resposta depois de processar a requisição (que ainda nem tinha sido enviada), o cliente sempre lia EOF na hora, imprimia "sem resposta do servidor" e removia seu próprio FIFO antes do servidor conseguir respondê-lo.

Correção: o cliente passou a abrir seu FIFO de resposta em modo `O_RDWR` em vez de `O_RDONLY` — o próprio processo cliente conta então como seu escritor,

garantindo que o `read()` bloqueie de verdade até a resposta real do servidor chegar (a mesma técnica já usada no lado do servidor para o FIFO de requisições, evitando o problema simétrico).

6.3 Tabela de desempenho e integridade

Dados brutos completos em `docs/resultados.md`, gerados por `testes/coletar_resultados.sh`:

Threads no pool	K (clientes concorrentes)	Tempo total (s)	Integridade
1	10	0.034	íntegro
1	50	0.030	íntegro
1	100	0.059	íntegro
2	10	0.014	íntegro
2	50	0.031	íntegro
2	100	0.053	íntegro
4	10	0.014	íntegro
4	50	0.029	íntegro
4	100	0.054	íntegro
8	10	0.015	íntegro
8	50	0.030	íntegro
8	100	0.053	íntegro

12 de 12 cenários íntegros, incluindo o mais pesado (8 threads, 100 clientes concorrentes disparados literalmente ao mesmo tempo).

7. Análise e discussão dos resultados finais

O resultado mais importante do trabalho não é um número de tempo, e sim a ausência total de corrupção de dado em todas as 12 combinações de carga testadas, incluindo o cenário de maior concorrência. Isso confirma que a

exclusão mútua implementada em `banco.c` cumpre sua função: nenhuma condição de corrida do tipo "check-then-act" (verificar se o `id` existe e só depois inserir) escapou para fora da seção crítica, mesmo com 8 threads competindo pelo mesmo mutex sob 100 requisições concorrentes.

O segundo resultado relevante é que o **tempo total não caiu de forma proporcional ao aumento do número de threads** — de 1 para 8 threads, o tempo para 100 `INSERT`s concorrentes ficou praticamente estável ($\sim 0,053$ – $0,059$ s). À primeira vista isso poderia parecer que o paralelismo não trouxe ganho, mas a explicação está no que realmente domina o tempo total nesta arquitetura: cada cliente é um **processo do sistema operacional inteiro** (não uma thread leve), então o custo de `fork` / `exec` de cada um dos K processos-cliente, somado à ida-e-volta pelo FIFO, é ordens de grandeza maior que o tempo gasto **dentro** da seção crítica (uma escrita num vetor em memória, que dura microssegundos). Ou seja: o pool de threads do servidor não é o gargalo deste desenho — o modelo "um processo por requisição" do lado cliente é. Essa é uma conclusão legítima de engenharia de concorrência, e ilustra por que otimizar exclusivamente o tamanho de um pool de threads sem antes identificar onde o tempo realmente é gasto costuma não produzir o ganho esperado.

Por fim, os dois bugs de concorrência encontrados e corrigidos durante os próprios testes (Seções 6.1 e 6.2) — ambos relacionados à semântica de FIFOs POSIX sob concorrência real, não a lógica de negócio — reforçam o objetivo da disciplina: a dificuldade de sistemas concorrentes corretos não está em escrever a lógica sequencial de cada operação, e sim em raciocinar corretamente sobre o que acontece quando múltiplos fluxos de execução independentes competem pelo mesmo recurso — seja o mutex do banco, seja o próprio canal de comunicação entre processos.